

WB Trust & Safety

Шпаргалка для HR/технического собеседования на Junior+ DS / ML Engineer

Темы: ML, антифрод, SQL, Python, production-мышление и готовые формулировки ответов.

Фокус: Wildberries / внутренний антифрод / Trust & Safety

Используй документ как конспект перед созвоном: сначала повтори готовые связные ответы в конце, затем пройди по каждому блоку.

WB Trust & Safety: шпаргалка для HR/тех собеседования DS

Документ составлен под позицию Junior+ DS / ML Engineer в команду внутреннего антифрода Wildberries: классический ML, Python, SQL, LLM/NLP, мультимодальные данные, end-to-end задачи от датасета до production.

Как пользоваться

- На HR-скрининге отвечай коротко, но показывай product thinking: не только метрики, а бизнес-стоимость ошибок.
- На техэтапе делай упор на антифродную специфику: leakage, time-based split, delayed labels, noisy labels, class imbalance, threshold tuning.
- Твой сильный угол: ты уже работал с WB-доменом, API, продавцами, рекламными и карточными данными.

1. ML

1.1. Train / valid / test split

Train / valid / test split - разделение данных на части. Train нужен для обучения модели, validation - для подбора гиперпараметров, признаков и threshold, test - для финальной честной проверки качества.

На интервью можно сказать: «Я делю данные минимум на train и test, а для настройки модели использую validation. Test нельзя трогать при подборе модели, иначе финальная оценка будет нечестной. В антифроде особенно важно делать split по времени, потому что будущие события не должны попадать в прошлое».

В антифроде лучше использовать time-based split:

```
train: январь - февраль
valid: март
test: апрель
```

Почему random split опасен: один и тот же пользователь, device, seller или fraud-группа могут попасть и в train, и в test. Тогда модель запомнит сущности, а не научится обобщать fraud-паттерны.

Типичные ошибки:

- random split для временных данных;
- подбор гиперпараметров на test;
- признаки считаются по всей истории, включая будущее;
- один user/device/seller попадает в train и test, создавая leakage.

Правильная фраза для WB: «Я бы обучал модель на прошлом периоде, валидировал на более свежем и тестировал на самом новом периоде. Для каждого признака проверял бы, был ли он доступен на момент скоринга».

1.2. Leakage

Data leakage - ситуация, когда в признаки попадает информация, которая в реальной системе была бы недоступна на момент предсказания. Модель получает нечестное преимущество: offline-метрика высокая, а в production качество падает.

Примеры плохих признаков для скоринга заказа в момент создания:

```
order_was_cancelled_later
refund_status
manual_moderation_result
days_until_ban
final_fraud_label
```

Виды leakage:

- target leakage - признак напрямую содержит ответ, например moderation_decision;
- time leakage - используются события из будущего, например complaints_after_order;
- aggregation leakage - агрегаты посчитаны по всей выборке, включая valid/test;
- split leakage - один и тот же fraud-кластер попал и в train, и в test.

Как бороться:

- проверять timestamp каждого признака;
- делать time-based split;
- агрегаты считать только по прошлым событиям;
- target encoding делать out-of-fold или строго по времени;
- смотреть feature importance: если наверху future_refund_count или moderation_result - это красный флаг.

Сильный ответ: «В антифроде я бы для каждого признака проверял point-in-time correctness. Если мы скорим заказ в момент создания, то нельзя использовать будущие возвраты, жалобы или последующие блокировки».

1.3. Cross-validation

Cross-validation - способ устойчиво оценить качество модели: данные несколько раз делятся на train/valid, модель обучается на разных фолдах, качество усредняется.

Основные виды:

- K-Fold - обычное разбиение на K частей;
- Stratified K-Fold - сохраняет долю классов в каждом фолде, полезно при дисбалансе;
- Group K-Fold - не дает объектам одной группы попасть в train и valid одновременно;
- TimeSeriesSplit / backtesting - проверка модели на будущих периодах.

Для антифрода обычный random K-Fold часто некорректен, потому что ломает временную структуру. Лучше использовать backtesting:

```
fold 1: train Jan -> valid Feb
fold 2: train Jan-Feb -> valid Mar
fold 3: train Jan-Mar -> valid Apr
```

Ответ на интервью: «Если данные независимы, можно использовать StratifiedKFold. Но для антифрода я бы предпочел time-based validation, потому что production всегда применяет модель к будущим событиям».

1.4. ROC-AUC vs PR-AUC

ROC-AUC показывает, насколько хорошо модель ранжирует положительные объекты выше отрицательных. ROC-кривая использует TPR и FPR.

```
TPR = TP / (TP + FN)
FPR = FP / (FP + TN)
```

PR-AUC строится по precision и recall:

```
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
```

Главное отличие: ROC-AUC может выглядеть слишком оптимистично при сильном дисбалансе, а PR-AUC лучше показывает качество на редком положительном классе.

В антифроде fraud может быть 0.1-1% объектов. Модель может иметь ROC-AUC = 0.95, но PR-AUC = 0.08. Это означает: ранжирование вроде неплохое, но среди срабатываний слишком много нормальных объектов.

Что сказать: «В антифроде я бы смотрел не только ROC-AUC, но и PR-AUC, precision@k, recall@k, recall при фиксированном precision или fixed FPR. Для редкого fraud-класса PR-AUC обычно информативнее».

1.5. Precision, Recall, F1

Базовые обозначения:

TP - модель сказала fraud, и это fraud
FP - модель сказала fraud, но это normal
TN - модель сказала normal, и это normal
FN - модель сказала normal, но это fraud

Precision отвечает на вопрос: «Среди всех объектов, которые модель назвала fraud, какая доля реально fraud?»

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall отвечает на вопрос: «Какую долю всех fraud-объектов модель нашла?»

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1 - гармоническое среднее precision и recall:

$$\text{F1} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

Для антифрода выбор зависит от действия:

- автоматическая блокировка - нужен высокий precision;
- ручная проверка - можно снизить precision ради большего recall;
- risk scoring - важно хорошее ранжирование и top-k качество.

Главная фраза: «Я бы не выбирал threshold 0.5 автоматически. Threshold нужно подбирать по бизнес-ограничениям: сколько кейсов можно отправить на проверку, какой допустимый false positive rate и какая стоимость пропущенного fraud».

1.6. CatBoost / LightGBM / XGBoost

Все три библиотеки реализуют градиентный бустинг над деревьями решений. Бустинг строит деревья последовательно: каждое новое дерево исправляет ошибки предыдущих.

CatBoost:

- хорошо работает с категориальными признаками;
- умеет обрабатывать категории почти из коробки;
- использует ordered target statistics, что снижает риск target leakage;
- часто дает сильный baseline на табличных данных.

LightGBM:

- быстрый;
- хорошо подходит для больших таблиц;
- эффективен по памяти;
- часто используется, когда данных очень много.

XGBoost:

- классический надежный вариант;

- хорошая регуляризация;
- часто требует более аккуратной обработки категорий.

Ответ для WB: «Для табличного антифрода я бы начал с CatBoost или LightGBM. CatBoost удобен, если много категорий: payment_type, device_type, region, seller_category. LightGBM хорош, когда очень большой объем данных и важна скорость».

Важные параметры:

```
learning_rate
iterations / n_estimators
depth / max_depth
subsample
l2_leaf_reg / reg_lambda
class_weights / scale_pos_weight
early_stopping_rounds
```

Типичные ошибки:

- оценивать только accuracy;
- не использовать early stopping;
- брать слишком глубокие деревья;
- не подбирать threshold;
- делать leakage в категориальных признаках.

1.7. Class imbalance

Class imbalance - сильный дисбаланс классов. В антифроде нормальных объектов обычно намного больше, чем fraud:

```
normal: 99.5%
fraud: 0.5%
```

Accuracy здесь почти бесполезна: модель может всегда говорить normal и получить 99.5% accuracy, но не поймать ни одного мошенника.

Методы борьбы:

- правильные метрики: PR-AUC, precision, recall, F1, precision@k, recall@k;
- class weights;
- oversampling fraud-класса;
- undersampling normal-класса;
- threshold tuning;
- cost-sensitive learning;
- top-k подход, если есть ограничение на ручную проверку.

Ответ: «В антифроде я бы сначала оценил базовую долю fraud. Потом использовал PR-AUC и top-k метрики, а при обучении попробовал class weights. Threshold выбирал бы не по 0.5, а по бизнес-ограничению».

1.8. Feature importance

Feature importance показывает, какие признаки сильнее всего влияют на предсказание модели.

Зачем нужна:

- понять, на чем модель принимает решения;
- найти leakage;
- объяснить модель бизнесу;
- отобрать полезные признаки;
- найти странные зависимости.

Виды:

- built-in importance в бустингах;
- permutation importance - перемешиваем признак и смотрим падение качества;
- SHAP - показывает вклад признаков в конкретное предсказание.

В антифродде feature importance особенно полезна для поиска leakage. Если самый важный признак - moderation_result, future_refund_count или status_after_check, значит модель использует будущую информацию.

Ответ: «Я бы смотрел feature importance не только для интерпретации, но и как sanity check. В антифродде подозрительно, если наверху оказываются признаки, появляющиеся после решения».

1.9. Target encoding / one-hot / embeddings

One-hot encoding создает бинарные колонки для категорий. Подходит для признаков с небольшим числом значений: payment_type, region, platform.

Target encoding заменяет категорию средним target по этой категории. Например, seller_id превращается в seller_fraud_rate. Это полезно для high-cardinality признаков, но опасно из-за leakage.

Правильный target encoding:

- считать только на train;
- использовать out-of-fold encoding;
- для временных данных учитывать только прошлые события;
- применять smoothing.

Embeddings - плотные векторные представления объектов: текста, изображения, товара, пользователя, продавца или графовой вершины. Они полезны для NLP, изображений, мультимодальных данных, LLM/NLP-фичей и high-cardinality сущностей.

Ответ: «Для малых категорий я бы использовал one-hot, для high-cardinality - target encoding или CatBoost encoding без leakage, для текстов и изображений - embeddings».

2. Антифрод

2.1. Supervised fraud detection

Supervised fraud detection - обучение модели на размеченных исторических данных:

X - признаки операции / пользователя / продавца
y - fraud / not fraud

Модель выдает вероятность fraud, дальше по threshold объект блокируется, отправляется на ручную проверку или получает risk-score.

Типичный pipeline:

1. определить fraud-сценарий и бизнес-действие;
2. собрать labels;
3. проверить задержку и качество labels;
4. построить признаки только из прошлого;
5. сделать time-based split;
6. обучить baseline CatBoost/LightGBM;
7. оценить PR-AUC, precision@k, recall@k;
8. разобрать ошибки;
9. добавить graph/text/image features;
10. настроить monitoring.

Ответ: «Я бы начал с табличного baseline: агрегаты по user/device/IP/payment/seller за окна 1/7/30 дней, CatBoost или LightGBM, time-based validation и метрики PR-AUC плюс precision@k/recall@k».

2.2. Anomaly detection

Anomaly detection - поиск объектов, которые сильно отличаются от нормального поведения. Не всегда нужна разметка fraud/not fraud.

Примеры аномалий:

- продавец внезапно получил 1000 заказов за час;
- один device используется сотнями аккаунтов;
- IP связан с большим числом пользователей;
- товар получил много отзывов за короткое время;
- резкий рост возвратов у seller.

Методы:

- статистические правила и threshold;
- Isolation Forest;
- One-Class SVM;
- autoencoders;
- clustering;
- graph anomaly detection.

Важно: аномалия не равна fraud. Резкий рост продаж может быть рекламной акцией, а не мошенничеством. Поэтому anomaly detection лучше использовать как сигнал для проверки или генерации гипотез.

Ответ: «Anomaly detection полезен, когда нет хорошей разметки или нужно искать новые схемы. Я бы использовал его как дополнение к supervised-модели и источник кандидатов на ручную проверку».

2.3. Graph features

Graph features - признаки, построенные на связях между сущностями. В антифроде это важно, потому что мошенники часто действуют группами.

Узлы графа:

user, device, IP, card, phone, address, seller, product, bank_account

Ребра:

user использовал device
user платил card
user заходил с IP
seller продает product
user оставил review

Признаки:

- accounts_per_device;
- users_per_ip;
- users_per_card;
- component_size;
- fraud_neighbors_rate;
- distance_to_known_fraud;
- PageRank / centrality;
- community id.

Ответ: «Я бы строил граф user-device-IP-card-seller-product и извлекал признаки вроде количества аккаунтов на device, доли fraud-соседей, размера компоненты и расстояния до известных fraud-узлов».

Осторожно: графовые признаки тоже должны строиться только по прошлым событиям, иначе будет leakage.

2.4. User / device / IP / payment / seller behavior

User features:

- account_age_days;
- orders_count_1d/7d/30d;
- cancel_rate;
- refund_rate;
- complaints_count;
- unique_addresses_count;
- unique_devices_count;
- avg_order_value;
- night_activity_rate.

Device features:

- accounts_per_device;
- orders_per_device;
- fraud_users_per_device;
- device_age_days;
- new_device_flag.

IP features:

- users_per_ip;
- orders_per_ip;
- countries_per_ip;
- ip_is_proxy / vpn;
- fraud_rate_ip.

Payment features:

- cards_per_user;
- users_per_card;
- failed_payments_count;
- payment_method;
- chargeback_rate;
- avg_payment_amount.

Seller features:

- seller_age_days;
- orders_count;
- GMV;
- refund_rate;
- complaint_rate;
- review_growth_rate;
- avg_rating;
- products_count;
- price_changes_count;
- ad_spend_growth.

Ответ: «Для каждой сущности можно считать агрегаты за 1/7/30 дней: количество действий, уникальные связанные сущности, долю возвратов, жалоб, fraud-соседей и скорость роста активности. Такие признаки часто дают сильный baseline».

2.5. Cold start

Cold start - ситуация, когда по новому объекту мало истории: новый пользователь, seller, device, card или product.

Как бороться:

- использовать признаки, доступные сразу: регион, тип устройства, канал регистрации, первые действия;
- использовать групповые статистики: fraud_rate_by_category, avg_refund_rate_by_region;
- использовать связи через device/IP/payment;
- применять отдельные правила для новых объектов;
- использовать текстовые/визуальные признаки товара или профиля seller.

Ответ: «Новый пользователь сам по себе не fraud, но новый пользователь на device, который уже связан с fraud-аккаунтами, - сильный сигнал. Для cold start я бы использовал признаки первых действий, групповые статистики и graph-связи».

2.6. Delayed labels

Delayed labels - правильная метка fraud/not fraud появляется не сразу. Например, fraud может выявиться через жалобу, возврат, chargeback или ручное расследование через 7-30 дней.

Проблема: если обучаться на слишком свежих данных, часть fraud еще не размечена и ошибочно попадет в negative class.

Решение - label maturity window. Если fraud обычно выявляется за 30 дней, то для обучения берем только события старше 30 дней.

Ответ: «Я бы посмотрел распределение времени до появления fraud-label и выбрал окно зрелости, например 14 или 30 дней. Это снижает риск ложных negatives в обучающей выборке».

2.7. Noisy labels

Noisy labels - ошибки в разметке: fraud помечен как normal или normal помечен как fraud.

В антифроде это часто, потому что:

- ручная проверка ошибается;
- не весь fraud обнаруживается;
- жалоба не всегда означает fraud;
- автоматическое правило может быть неточным;
- разные модераторы могут принимать разные решения.

Подход:

- разделять strong labels и weak labels;
- давать разный вес источникам разметки;
- анализировать спорные объекты;
- использовать human-in-the-loop;
- не считать свежие unlabeled объекты гарантированно normal.

Ответ: «Я бы различал confirmed fraud и weak signals. Например, подтвержденное расследование - strong label, а жалоба или автоматический флаг - weak label».

2.8. False positives vs false negatives

False positive - модель сказала fraud, но объект нормальный. Последствия: блокировка честного пользователя, потеря продавца, жалобы, плохой UX, нагрузка на поддержку.

False negative - модель сказала normal, но объект fraud. Последствия: финансовый ущерб, рост мошенничества, вред платформе.

В антифроде нельзя смотреть только на ML-метрики. Нужно понимать стоимость ошибок.

Матрица:

- автоматическая блокировка - очень высокий precision;
- ручная проверка - баланс recall и manageable precision;
- risk scoring - хорошее ранжирование;
- крупный финансовый fraud - высокий recall;
- блокировка seller - особенно высокий precision.

Ответ: «Для автоматической блокировки нужен высокий precision, чтобы не задевать честных пользователей. Для ручной проверки можно позволить больше FP, если нужно поднять recall».

2.9. Monitoring drift

Drift - изменение данных или связи признаков с target во времени.

Виды:

- data drift - изменилось распределение признаков;
- concept drift - изменилась связь между признаками и target;
- label drift - изменилась доля fraud.

Что мониторить:

- PR-AUC, precision@k, recall@k, если labels доступны;
- распределения признаков: PSI, KL divergence, KS-test, quantiles, missing rate;
- распределение score;
- долю объектов выше threshold;
- бизнес-метрики: ущерб, число блокировок, appeal rate, ручная нагрузка.

Ответ: «После вывода в production я бы мониторил не только ML-метрики, но и feature distribution, score distribution, долю срабатываний, appeal rate и бизнес-ущерб. В антифроде drift особенно важен, потому что мошенники адаптируются».

3. SQL

3.1. JOIN

JOIN объединяет строки из нескольких таблиц по условию.

Основные виды:

- INNER JOIN - только совпавшие строки;
- LEFT JOIN - все строки из левой таблицы плюс совпадения из правой;
- RIGHT JOIN - все строки из правой таблицы;
- FULL JOIN - все строки из обеих таблиц.

В feature engineering часто используется LEFT JOIN от основной таблицы событий, чтобы не потерять заказы без дополнительных данных.

Пример:

```
SELECT
    o.order_id,
    o.user_id,
```

```

    o.seller_id,
    o.amount,
    u.region,
    p.payment_type
FROM orders o
LEFT JOIN users u ON o.user_id = u.user_id
LEFT JOIN payments p ON o.order_id = p.order_id;

```

Типичные ошибки:

- INNER JOIN случайно удаляет строки;
- many-to-many JOIN размножает строки;
- правую таблицу не агрегировали до нужного уровня;
- неправильный ключ JOIN;
- NULL не обработаны.

3.2. GROUP BY

GROUP BY группирует строки и позволяет считать агрегаты: COUNT, SUM, AVG, MIN, MAX, COUNT DISTINCT.

Пример:

```

SELECT
    user_id,
    COUNT(*) AS orders_count,
    SUM(amount) AS total_amount,
    AVG(amount) AS avg_amount,
    COUNT(DISTINCT seller_id) AS unique_sellers
FROM orders
GROUP BY user_id;

```

Ответ: «GROUP BY используется для построения агрегированных признаков: количество заказов пользователя, средний чек, число уникальных продавцов, доля возвратов».

Ошибка: считать агрегаты по всей истории, включая будущее. Для антифрода надо считать только прошлые события относительно момента скоринга.

3.3. HAVING

HAVING фильтрует группы после GROUP BY. WHERE фильтрует строки до группировки.

Пример:

```

SELECT
    user_id,
    COUNT(*) AS orders_count
FROM orders
WHERE created_at >= '2025-01-01'
GROUP BY user_id
HAVING COUNT(*) > 10;

```

Ответ: «WHERE - взять заказы за последние 7 дней, HAVING - оставить пользователей, у которых больше 20 заказов за эти 7 дней».

3.4. Оконные функции: ROW_NUMBER, LAG, SUM() OVER

Оконные функции считают значения по группе строк, но не схлопывают строки как GROUP BY.

ROW_NUMBER нумерует строки внутри группы:

```

SELECT *
FROM (
    SELECT
        o.*,
        ROW_NUMBER() OVER (

```

```

        PARTITION BY user_id
        ORDER BY created_at
    ) AS rn
FROM orders o
) t
WHERE rn = 1;

```

LAG берет значение из предыдущей строки:

```

SELECT
    user_id,
    order_id,
    created_at,
    created_at - LAG(created_at) OVER (
        PARTITION BY user_id
        ORDER BY created_at
    ) AS time_since_prev_order
FROM orders;

```

SUM() OVER считает накопительную или оконную сумму:

```

SELECT
    user_id,
    order_id,
    amount,
    SUM(amount) OVER (
        PARTITION BY user_id
        ORDER BY created_at
    ) AS cumulative_amount
FROM orders;

```

Ответ: «Оконные функции позволяют строить признаки по истории объекта: номер заказа пользователя, время с предыдущего заказа, накопительная сумма, активность за окно».

3.5. CTE

CTE - Common Table Expression, временное именованное выражение через WITH. Оно делает сложный SQL читаемым.

Пример:

```

WITH user_orders AS (
    SELECT
        user_id,
        COUNT(*) AS orders_count,
        SUM(amount) AS total_amount
    FROM orders
    GROUP BY user_id
)
SELECT *
FROM user_orders
WHERE orders_count > 10;

```

Ответ: «Я бы через CTE считал user features, device features, seller features, а потом объединял их в один датасет для модели».

3.6. Признаки за окна 1 день / 7 дней / 30 дней

В антифродде важна активность за разные периоды:

- 1 день - резкие всплески;
- 7 дней - недельные паттерны;
- 30 дней - стабильная история.

Примеры:

- orders_count_1d/7d/30d;
- amount_sum_1d/7d/30d;

- unique_sellers_1d/7d/30d;
- users_per_device_1d/7d/30d;
- refund_rate_30d;
- complaints_count_30d.

Ключевой момент: признаки должны считаться только по прошлому.

```
o2.created_at < o.created_at
```

Ответ: «Я бы строил rolling features за 1/7/30 дней и обязательно следил, чтобы в них попадали только события до текущего момента, иначе будет leakage».

4. Python

4.1. pandas

pandas - библиотека для работы с табличными данными. Основные структуры: DataFrame и Series.

Что нужно уметь:

- загрузка данных: read_csv, read_parquet;
- EDA: head, info, describe, shape;
- фильтрация;
- groupby и agg;
- merge;
- работа с пропусками;
- работа с датами;
- построение признаков.

Пример агрегатов:

```
user_agg = df.groupby('user_id').agg(
    orders_count=('order_id', 'count'),
    total_amount=('amount', 'sum'),
    avg_amount=('amount', 'mean'),
    unique_sellers=('seller_id', 'nunique')
).reset_index()
```

Ответ: «В pandas я обычно делаю EDA, очистку данных, агрегации, merge разных источников, построение признаков и подготовку датасета для модели. Для очень больших данных логику лучше переносить в SQL/Spark».

4.2. sklearn

scikit-learn - основная библиотека для классического ML в Python.

Что есть:

- train_test_split;
- preprocessing;
- Pipeline;
- ColumnTransformer;
- cross-validation;
- модели;
- метрики;
- GridSearchCV / RandomizedSearchCV.

Пример baseline:

```
from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import average_precision_score
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

model = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42
)
model.fit(X_train, y_train)
preds = model.predict_proba(X_test)[: , 1]
print(average_precision_score(y_test, preds))

```

Ответ: «В sklearn я использую split, preprocessing, Pipeline, cross-validation и метрики. Для антифрода sklearn удобен для baseline и структуры экспериментов, а для сильной табличной модели часто беру CatBoost/LightGBM».

4.3. CatBoost

CatBoost - градиентный бустинг, хорошо работающий с табличными данными и категориальными признаками.

Плюсы:

- сильный baseline;
- категории из коробки;
- early stopping;
- class weights;
- feature importance;
- хорошо подходит для fraud detection.

Пример:

```

from catboost import CatBoostClassifier

cat_features = ['payment_type', 'region', 'device_type']

model = CatBoostClassifier(
    iterations=1000,
    learning_rate=0.05,
    depth=6,
    loss_function='Logloss',
    eval_metric='PRAUC',
    class_weights=[1, 20],
    random_seed=42,
    verbose=100
)

model.fit(
    X_train, y_train,
    cat_features=cat_features,
    eval_set=(X_valid, y_valid),
    early_stopping_rounds=100
)

```

Ответ: «CatBoost я бы выбрал как baseline для табличного антифрода, особенно если много категориальных признаков. После обучения отдельно подбирал бы threshold по бизнес-метрике».

4.4. numpy

numpy - библиотека для быстрых численных вычислений и работы с массивами.

Важные темы:

- ndarray;

- векторизация;
- broadcasting;
- индексация и boolean masks;
- mean/std/quantile;
- matrix operations.

Пример:

```
import numpy as np

arr = np.array([1, 2, 3])
result = arr ** 2
mask = arr > 1
```

Ответ: «numpy я использую для массивов, векторных операций, численных вычислений и подготовки данных. Он быстрее Python-циклов, потому что операции выполняются в оптимизированном C-коде».

4.5. copy vs deepcopy

В Python переменные часто хранят ссылку на объект.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
```

copy.copy делает поверхностную копию: внешний объект новый, вложенные объекты общие.

deepcopy рекурсивно копирует вложенные объекты.

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
b[0].append(100)
print(a) # [[1, 2], [3, 4]]
```

Ответ: «copy.copy делает shallow copy, а copy.deepcopy копирует объект рекурсивно. Это важно для списков, словарей, конфигов и вложенных структур, чтобы случайно не изменить исходный объект».

4.6. Mutable / immutable

Mutable objects можно менять на месте:

- list;
- dict;
- set;
- DataFrame;
- numpy array.

Immutable objects нельзя менять на месте:

- int;
- float;
- str;
- tuple;
- bool;
- None.

Опасный пример:

```
def add_item(x=[]):
```

```
x.append(1)
return x
```

Default argument создается один раз, поэтому список будет переиспользоваться между вызовами.

Правильно:

```
def add_item(x=None):
    if x is None:
        x = []
    x.append(1)
    return x
```

Ответ: «Mutable objects можно менять на месте, immutable - нельзя. Поэтому со списками и словарями нужно быть аккуратным: функция может изменить исходный объект».

4.7. Generators / list comprehensions

List comprehension сразу создает список:

```
squares = [x ** 2 for x in range(10)]
```

Generator expression создает значения лениво, по одному:

```
gen = (x ** 2 for x in range(10))
```

Генератор через yield:

```
def read_rows(path):
    with open(path) as f:
        for line in f:
            yield line.strip()
```

Когда использовать generator:

- большие данные;
- потоковая обработка;
- чтение логов построчно;
- когда нельзя держать все в памяти.

Ответ: «List comprehension хранит весь список в памяти, generator вычисляет элементы по мере итерации. Для больших логов и потоковой обработки генераторы полезнее».

4.8. Basic OOP

OOP - объектно-ориентированное программирование.

Основные идеи:

- class - шаблон объекта;
- object - экземпляр класса;
- encapsulation - данные и методы вместе;
- inheritance - наследование поведения;
- polymorphism - общий интерфейс для разных объектов;
- abstraction - скрытие деталей реализации.

Пример ML pipeline:

```
class FraudPipeline:
    def __init__(self, model, threshold):
        self.model = model
        self.threshold = threshold

    def fit(self, X_train, y_train):
        self.model.fit(X_train, y_train)

    def predict_scores(self, X):
```



```
return self.model.predict_proba(X)[: , 1]

def predict_labels(self, X):
    scores = self.predict_scores(X)
    return (scores >= self.threshold).astype(int)
```

Ответ: «В ООР класс описывает сущность, объект - конкретный экземпляр. Инкапсуляция хранит состояние и методы вместе, наследование переиспользует код, полиморфизм позволяет работать с разными моделями через общий интерфейс, например fit/predict_proba».

5. Готовые связанные ответы

Как бы ты строил антифрод-модель?

«Сначала я бы уточнил fraud-сценарий и бизнес-действие: блокировка, ручная проверка или risk-score. Потом собрал бы labels и проверил их задержку и качество. Дальше сделал бы time-based split, чтобы обучаться на прошлом и проверяться на будущем.

Для baseline построил бы табличные признаки по user/device/IP/payment/seller за окна 1/7/30 дней. В качестве модели взял бы CatBoost или LightGBM, потому что они хорошо работают с табличными данными и категориями. Из метрик смотрел бы PR-AUC, precision@k, recall@k, а threshold выбирал бы по стоимости false positives и false negatives. После этого анализировал бы ошибки, добавлял graph features, текстовые признаки или embeddings, а в production мониторил бы drift, качество данных и бизнес-метрики».

Какая главная сложность в антифроде?

«Главная сложность в том, что fraud редкий, labels появляются с задержкой, разметка шумная, а мошенники адаптируются. Поэтому важно не только обучить модель, но и правильно построить датасет без leakage, выбрать метрики под бизнес, мониторить drift и регулярно обновлять признаки/модель».

Почему не accuracy?

«Потому что при fraud rate 0.5% модель может всегда предсказывать normal и получить 99.5% accuracy, но не поймать ни одного мошенника. Поэтому лучше использовать PR-AUC, precision, recall, precision@k и бизнес-метрики».

Почему CatBoost?

«В антифроде много табличных и категориальных признаков: тип оплаты, регион, устройство, seller category, разные ID и агрегаты. CatBoost хорошо работает с такими данными, умеет обрабатывать категориальные признаки и быстро дает сильный baseline. Но для больших объемов я бы также сравнил LightGBM».

Как избежать leakage?

«Для каждого признака нужно понимать, был ли он известен на момент скоринга. Все агрегаты надо считать только по прошлым событиям. Split лучше делать по времени. Target encoding - только out-of-fold или с учетом времени. И отдельно проверять самые важные признаки модели: если наверху feature importance оказался признак из будущего, это красный флаг».

Как выбрать threshold?

«Не надо автоматически брать 0.5. Threshold выбирается по бизнес-ограничениям: сколько объектов можно отправить на ручную проверку, какой допустимый false positive rate, насколько

дорогой пропущенный fraud. Для автоматической блокировки нужен высокий precision, для ручной проверки можно делать упор на recall».

6. Главное запомнить

1. В антифроде почти всегда class imbalance, поэтому accuracy не подходит.
2. Главная опасность - leakage, особенно из будущих событий и неправильных агрегатов.
3. Split лучше делать по времени, потому что production работает на будущем.
4. Метрики выбираются от бизнес-действия: блокировка, ручная проверка, risk scoring.
5. Сильный baseline - CatBoost/LightGBM + агрегаты по user/device/IP/payment/seller за окна 1/7/30 дней.